ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ

УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра «Програмної інженерії»

Звіт

з лабораторної роботи №1

з дисципліни «Програмування мовою Dart»

Виконали:

ст. гр. ПЗПІ-23-10

Шахмаєв Д.О.

Перевірив:

асистент каф. ПІ

Баличев М. Д.

2026

МЕТА РОБОТИ ТА ЗАВДАННЯ ЗА ВАРІАНТОМ

1.1 Мета роботи Отримання практичних навичок оголошення функцій із різними видами параметрів, побудови конвеєрів обробки колекцій засобами Iterable, а також обробки рядків, дат і регулярних виразів мовою Dart. Набуття досвіду роботи із системою контролю версій Git та дотримання стандартів якості коду (Effective Dart).

1.2 Завдання за варіантом Предметна область за варіантом: Щоденник тренувань. Атрибути сутності: тренування: дата, вид активності, тривалість, дистанція, пульс, самопочуття

Основні вимоги до виконання:

1. Розробити консольну програму для розбору, перевірки та агрегації набору записів (не менше 15 рядків, серед яких є пошкоджені).
2. Дані повинні представлятися колекціями та записами (Record), обробка має виконуватись виключно функціями, без використання класів.
3. Реалізувати конвеєри обробки (where → map → fold).
4. Написати функцію-валідатор для перевірки діапазонів значень.
5. Реалізувати групування записів та обчислення агрегованих значень.
6. Застосувати замикання (closure).
7. Використати регулярні вирази з іменованими групами для перевірки та маскування текстового поля.
8. Виконати арифметику дат (обчислення віку альбому в днях).
9. Забезпечити відсутність зауважень аналізатора (dart analyze) та покрити код базовими модульними тестами.

Посилання на репозиторій Git:

https://github.com/NureShakhmaievDenys/dart

2 ОСНОВНА ЧАСТИНА. ОБҐРУНТУВАННЯ ПРИЙНЯТИХ РІШЕНЬ

Відповідно до вимог лабораторної роботи, об’єктно-орієнтований підхід у програмі не використовувався. Дані про тренування представлені іменованим записом Training, оголошеним через typedef. Він містить дату тренування, вид активності, тривалість, дистанцію, пульс і самопочуття.

```
typedef Training = ({
  DateTime date,
  String activity,
  int durationMinutes,
  double distanceKilometers,
  int heartRate,
  String wellbeing,
});
typedef ProcessingResult = ({List<Training> items, List<String> errors});
```

Запис обрано замість класу, оскільки в межах першої лабораторної роботи необхідно зосередитися на функціях, колекціях і типах даних Dart. Training забезпечує статичну типізацію, читабельні імена полів і не потребує створення окремого класу з конструкторами та методами.

Результат обробки всіх рядків подається записом ProcessingResult, що містить окремі списки коректних записів і повідомлень про помилки. Такий підхід дозволяє не переривати виконання програми через один пошкоджений рядок.

Код поділено між двома каталогами. У bin/ розміщено лише точку входу main, яка запускає обробку та виводить готовий звіт. У lib/ міститься основна логіка: розбір рядків, валідація, групування, сортування, форматування звіту та допоміжні функції. Така структура відділяє інтерфейс консольної програми від логіки предметної області та спрощує подальше повторне використання коду.

2.2 Вхідні дані, розбір і валідація

Вхідні дані подано константним списком CSV-подібних рядків. Роздільником полів є символ ;. У наборі міститься 18 рядків, з яких 15 є коректними, а три навмисно пошкоджені: один містить нечислове значення тривалості, у другому пропущене поле, а третій має некоректну дату.

Функція parseTraining спочатку ділить рядок методом split, перевіряє кількість полів, а потім виконує безпечні перетворення типів за допомогою int.tryParse, double.tryParse і DateTime.tryParse.

```
44    final date = DateTime.tryParse(fields[0]);
45    final duration = int.tryParse(fields[2]);
46    final distance = double.tryParse(fields[3]);
47    final heartRate = int.tryParse(fields[4]);
48
```

Для очікуваних помилок введення не використовується конструкція try/catch. Якщо перетворення не виконалося, функція повертає запис із null у полі результату та поясненням помилки. Це дає змогу продовжити обробку інших даних.

Особливістю DateTime.tryParse є те, що деякі некоректні дати можуть бути нормалізовані. Наприклад, значення місяця 13 може бути перетворене на дату наступного року. Тому після розбору додатково виконується порівняння початкового рядка з нормалізованим значенням дати.

```
49    if (date == null || date.toIso8601String().substring(0, 10) != fields[0]) {
50      return (item: null, error: 'некоректна дата "${fields[0]}"');
51    }
```

Для перевірки бізнес-правил реалізовано окрему функцію validateTraining. Вона перевіряє допустимі межі року, тривалості, дистанції, пульсу та наявність значення самопочуття. Функція повертає список усіх виявлених порушень, а не лише першу помилку.

2.3 Обробка колекцій і агрегація даних

Основна обробка колекцій побудована засобами Iterable. Для відбору, перетворення та згортання даних використано методи where, map і fold.

Наприклад, сумарна тривалість тренувань обчислюється так:

```
158     final totalMinutes = items
159         .map((item) => item.durationMinutes)
160         .fold(0, (sum, value) => sum + value);
```

Спочатку map перетворює список тренувань на послідовність тривалостей, а fold накопичує їхню суму. Аналогічно обчислюються загальна дистанція та середній пульс.

Для визначення кількості тренувань із високим пульсом використано фільтрацію:

```
164     final highPulseCount = items.where((item) => item.heartRate >= 150).length;
```

Такий стиль є декларативним: код описує необхідний результат, а не покрокове керування змінними-накопичувачами в циклах.

2.4 Групування та сортування

Тренування групуються за видом активності: біг, йога, плавання, велосипед, силове тренування та ходьба. Для цього застосовано тип Map<String, List<Training>>. Ключем є основний вид активності, а значенням — список усіх відповідних записів.

```
Map<String, List<Training>> groupByActivityKind(Iterable<Training> items) {
  return items.fold(<String, List<Training>>{}, (groups, item) {
    final kind = _activityPattern
        .firstMatch(item.activity)!
        .namedGroup('kind')!;
    groups.putIfAbsent(kind, () => <Training>[]).add(item);
    return groups;
  });
}
```

Для кожної групи у звіті обчислюється середня тривалість тренувань.

Сортування реалізовано за двома критеріями: тривалістю або дистанцією. Для цього створено перелік SortCriterion. Напрям сортування передається іменованим параметром ascending, тому виклик функції залишається зрозумілим:

```
136  String buildReport(
137    List<Training> items, {
138    List<String> invalidLines = const [],
139    SortCriterion sortBy = SortCriterion.duration,
140    bool ascending = false,
141    String separator = ' | ',
142    DateTime? reportDate,
143  }) {
```

2.5 Використання замикання

У функції processLines використано замикання-лічильник. Локальна змінна processedCount захоплюється вкладеною функцією countProcessed.

```
95    var processedCount = 0;
96    int countProcessed() => ++processedCount;
```

Під час обробки кожного вхідного рядка функція countProcessed викликається всередині map. Замикання зберігає доступ до змінної processedCount навіть під час виконання вкладеної функції. Після завершення обробки перевіряється, чи збігається кількість опрацьованих рядків із розміром початкового списку.

2.6 Регулярні вирази та обробка рядків

Для перевірки формату виду активності застосовано регулярний вираз у сирому рядку r'...'. Він містить іменовані групи kind і detail.

```
34  final _activityPattern = RegExp(
35    r'^(?<kind>[А-Яа-яІіЇїЄєA-Za-z]+)(?:\s+(?<detail>[А-Яа-яІіЇїЄєA-Za-z]+))?$',
36  );
```

Іменовані групи роблять код зрозумілішим, оскільки замість числових індексів використано семантичні назви. Наприклад, для рядка біг ранок група kind містить біг, а група detail — ранок.

Для маскування уточнення виду активності використано метод replaceAllMapped. У звіті, наприклад, значення біг темповий подається як біг ********.

```
127  String maskActivityDetail(String activity) {
128    return activity.replaceAllMapped(_activityPattern, (match) {
129      final activityMatch = match as RegExpMatch;
130      final kind = activityMatch.namedGroup('kind')!;
131      final detail = activityMatch.namedGroup('detail');
132      return detail == null ? kind : '$kind ${'*' * detail.length}';
133    });
134  }
```

Для формування великого текстового звіту застосовано StringBuffer, оскільки багаторазова конкатенація рядків оператором + або += менш ефективна та гірше масштабується.

2.7 Робота з датами

Кожен запис містить дату тренування. Для визначення давності запису використано метод DateTime.difference, який повертає об’єкт типу Duration.

```
196      final ageInDays = referenceDate.difference(item.date).inDays;
```

У звіті для кожного тренування виводиться кількість днів від дати тренування до дати формування звіту. Це демонструє застосування DateTime для роботи з датами та Duration для отримання різниці між ними.

2.8 Залежності та якість коду

У програмі не використано залежностей для основної логіки. Усі необхідні можливості надає стандартна бібліотека Dart: колекції, DateTime, Duration, RegExp і StringBuffer.

У секції dev_dependencies підключено пакет lints. Він потрібен лише під час розробки та вмикає рекомендований набір правил статичного аналізатора Dart через файл analysis_options.yaml. Альтернативою було б не використовувати додаткові правила, однак пакет lints допомагає дотримуватися рекомендацій Effective Dart і виявляти потенційні проблеми до запуску програми.

Перед завершенням роботи виконано форматування командою dart format і статичний аналіз командою dart analyze. Аналізатор не виявив помилок або зауважень.

3 РЕЗУЛЬТАТИ ВИКОНАННЯ

3.1 Перевірка форматування та статичного аналізу

Перед запуском програми було виконано форматування вихідного коду командою:

dart format --set-exit-if-changed .

Команда підтвердила, що всі файли проєкту відповідають стандартному стилю форматування Dart.

Для перевірки типів, імпортів, правил стилю та потенційних помилок було виконано статичний аналіз:

dart analyze

У результаті отримано повідомлення:

Analyzing lab1...

No issues found!

Це підтверджує, що програма компілюється без помилок і не містить зауважень аналізатора. У проєкті використано рекомендований набір правил пакета lints через файл analysis_options.yaml.

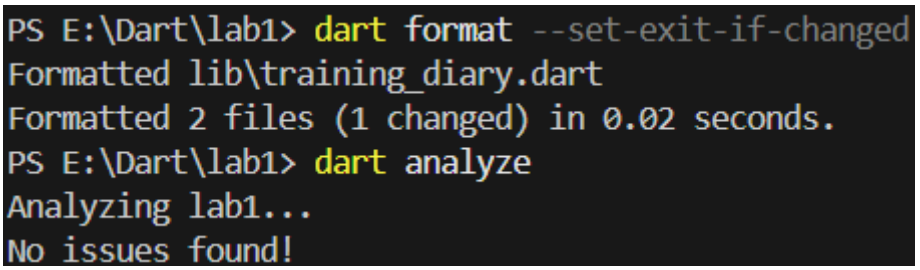


Рисунок 1 — Результат виконання команд dart format і dart analyze

3.2 Результат запуску програми

Після запуску командою:

dart run

програма успішно обробила 18 вхідних рядків. Із них 15 записів про тренування були визнані коректними, а 3 записи — відхилені через помилки формату.

Програма виявила такі пошкоджені дані:

1. нечислове значення тривалості тренування;
2. недостатню кількість полів у рядку;
3. некоректну дату.

Для коректних записів сформовано підсумковий звіт. Він містить загальну тривалість усіх тренувань, сумарну дистанцію, середній пульс, кількість тренувань із пульсом від 150 ударів за хвилину, групування за видом активності та відсортований список записів.

У блоці групування показано кількість записів і середню тривалість для кожного виду активності. Наприклад, програма визначає окремі групи для бігу, йоги, плавання, велосипеда, силових тренувань і ходьби.

Записи сортуються за дистанцією у спадному порядку. Для тренувань, що мають додаткове уточнення виду активності, наприклад біг ранок або біг темповий, уточнення маскується регулярним виразом.

Наприкінці звіту виводиться перелік відхилених рядків із поясненням причини помилки. Завдяки цьому користувач може визначити, які дані потрібно виправити.

```
PS E:\Dart\lab1> dart run
Building package executable...
Built lab1_training_diary:lab1_training_diary.
ЩОДЕННИК ТРЕНУВАНЬ
Коректних записів: 15; відхилених: 3
Загальна тривалість: 767 хв
Загальна дистанція: 90.6 км
Середній пульс: 130.8 уд/хв
Тренувань з пульсом від 150: 3

Групування за видом активності:
біг: 5 запис(ів), середня тривалість 38.8 хв
йога: 2 запис(ів), середня тривалість 47.5 хв
плавання: 2 запис(ів), середня тривалість 37.5 хв
велосипед: 2 запис(ів), середня тривалість 77.5 хв
силове: 2 запис(ів), середня тривалість 56.5 хв
ходьба: 2 запис(ів), середня тривалість 67.5 хв

Записи (distance, за спаданням):
2026-10-18 | велосипед | 90 хв | 27.5 км | 145 уд/хв | давність 14 дн.
2026-10-06 | велосипед | 65 хв | 18.4 км | 142 уд/хв | давність 26 дн.
2026-10-22 | біг ******** | 47 хв | 8.1 км | 158 уд/хв | давність 10 дн.
2026-10-02 | біг ***** | 42 хв | 7.2 км | 148 уд/хв | давність 30 дн.
2026-10-08 | біг ********* | 38 хв | 6.0 км | 164 уд/хв | давність 24 дн.
2026-10-12 | ходьба | 75 хв | 5.8 км | 105 уд/хв | давність 20 дн.
2026-10-28 | біг ****** | 36 хв | 5.5 км | 144 уд/хв | давність 4 дн.
2026-10-14 | біг ***** | 31 хв | 5.0 км | 151 уд/хв | давність 18 дн.
2026-10-24 | ходьба | 60 хв | 4.7 км | 101 уд/хв | давність 8 дн.
2026-10-16 | плавання | 40 хв | 1.3 км | 136 уд/хв | давність 16 дн.
2026-10-05 | плавання | 35 хв | 1.1 км | 132 уд/хв | давність 27 дн.
2026-10-03 | йога | 50 хв | 0.0 км | 92 уд/хв | давність 29 дн.
2026-10-10 | силове | 55 хв | 0.0 км | 126 уд/хв | давність 22 дн.
2026-10-20 | йога | 45 хв | 0.0 км | 88 уд/хв | давність 12 дн.
2026-10-26 | силове | 58 хв | 0.0 км | 130 уд/хв | давність 6 дн.
```

Рисунок 2 - Знімок екрана роботи програми 1 частина

```
Відхилені записи:
"2026-10-29;плавання;не число;1.0;128;добре": некоректне числове поле
"2026-10-30;велосипед;50;12.0;140": очікується 6 полів, отримано 5
"2026-13-01;йога;40;0;90;добре": некоректна дата "2026-13-01"
```

Рисунок 3 - Знімок екрана роботи програми 2 частина

4 ВИСНОВКИ

У ході виконання лабораторної роботи №1 було розроблено консольну програму мовою Dart для обробки щоденника тренувань відповідно до варіанта №7. Програма приймає набір CSV-подібних рядків, безпечно розбирає їх, виявляє пошкоджені записи, перевіряє допустимість значень і формує підсумковий текстовий звіт.

Під час роботи було закріплено навички використання функцій із позиційними та іменованими параметрами, записів Record, колекцій List і Map, а також методів where, map і fold для декларативної обробки даних. Було реалізовано групування тренувань за видом активності, сортування за різними критеріями та обчислення агрегованих показників: загальної тривалості, дистанції та середнього пульсу.

Також було застосовано замикання для підрахунку оброблених рядків, регулярний вираз з іменованими групами для перевірки й маскування текстових даних, StringBuffer для формування звіту та класи DateTime і Duration для обчислення давності тренувань у днях.

Програма коректно обробляє помилки у вхідних даних без аварійного завершення, оскільки для перетворення значень використано tryParse замість try/catch. Код відформатовано засобами Dart і перевірено командою dart analyze, яка не виявила зауважень. Поставлену мету лабораторної роботи досягнуто.